Compiler Optimizations for Attention and Functional KV Caches

CS6886: Systems Engineering for Deep Learning 1 November 2026

Nishanth Senthil Kumar (EE23B049) IIT Madras

Main result. On a Tesla T4, winner-cache factoring lowers a pretrained two-candidate step's IREE latency by **26.46% at context 2048**. Sparse synthetic cache forwarding reduces measured DRAM bytes by **99.92%**. These are different workloads; full-model DRAM reduction is not consistent.

1 Problem and baseline investigation

- **GQA and caching.** A KV cache stores earlier tokens' keys and values. With H_q query and H_{kv} KV heads, GQA shares each KV head across $g = H_q/H_{kv}$ query heads: $h_{kv} = \lfloor h_q/g \rfloor$ [1, 2].
- **Initial hypothesis.** Explicit `repeat_interleave`/reshape can expand K/V tensors. A C++ rewrite improved the original default CPU case **18.47 to 9.94 μs (1.86×)**, but stronger existing IREE settings remove the tested expansion too.
- **Deciding settings.** Ablation across head ratios and shapes established that both flags below were needed for the tested grouped cases; O3 alone was insufficient [13, 16].

```
--iree-opt-generalize-matmul
--iree-dispatch-creation-enable-aggressive-reshape-movement
```

The subsequent study targets missed functional-cache patterns.

2 Main transformations

Let C be an immutable history, U a t-row update at position p, and $C' = \mathrm{update}(C, p, U)$.

$$\underbrace{C'[i]}_{\text{read after update}} \longrightarrow \begin{cases} U[i-p], & p \le i < p+t, \\ C[i], & \text{otherwise.} \end{cases}$$

Read forwarding: sparse/gather consumers access the base or patch directly. An unobserved updated tensor may disappear; a returned full cache still needs its required representation.

$$\underbrace{\mathrm{select}(c, \mathrm{concat}(C, a), \mathrm{concat}(C, b))}_{\text{select full candidate caches}} \longrightarrow \underbrace{\mathrm{concat}(C, \mathrm{select}(c, a, b))}_{\text{select small rows, then commit}}.$$

Winner factoring: same history/append axis, scalar Boolean c. The model has **60 matches** (K/V in 30 layers); ordinary decode has **zero**. Candidate attention computations remain.

3 Prior work and implemented extensions

Transformation	Implementation / intended use	Boundary / prior work
Read/gather forwarding	Base-or-patch reads for forked sparse attention; normalize gather indices.	Established read-over-write algebra [3, 4]; full cache observations remain.
Winner commit factoring	Select small rows before a common-history append / equal-region update.	Custom branching step; standard selection algebra, not complete beam/speculative decoding.
Update coalescing	Merge adjacent small writes before forwarding multi-token chains.	Reject gaps, overlaps or observed intermediate versions. Classical update simplification.
Snapshot protection	Preserve returned old caches and supported borrowed-buffer ownership.	Correctness repair; sharing/COW precedents [5, 6]. Invalid outputs are excluded.
Projection / partial RoPE	$q(CW_K)^T = (qW_K^T)C^T$; project content separately from rotary coordinates.	MLA absorption / decoupled RoPE are prior art [7, 8]; require a numerical contract.
Native-attention guard	Preserve complete feature reduction before online softmax; decomposed fallback for failures.	Established online reduction [10, 11]; early upstream decomposition is a CPU alternative.
Stable trig reduction	Bounded FP64 argument reduction inside small FP32 sine/cosine kernels.	Repair for $32 < \lvert x \rvert \le 8192$, kernel size ≤ 4096. Not full Payne–Hanek or an all-input accuracy proof [9].

4 Compiler integration



Standalone MLIR 19 driver [12], with a standard-IR bridge to IREE 3.13/LLVM 24 before dispatch creation. Type, shape, SSA identity, indexing, use and lowering-state guards reject unsupported patterns. No new GPU kernel or complete patched in-tree build.

5 Evaluation protocol

Standalone C++ MLIR 19 rewriting precedes IREE 3.13/LLVM 24 compilation (`89ece87b8985`) [12]. GPU: Tesla T4, driver 580.82.07, FP32, TF32 off. SmolLM2-135M-Instruct [14], checkpoint `12fd25f77366`: 30 layers, 9Q/3KV heads, width 64. Both IREE variants use O3, both GQA flags, identical split-argmax settings, trig repair and f64-preservation flags. Timing excludes compilation, prefill and upload: resident outputs, context synchronization, randomized order, 31 samples/repetition; median of nine repetition medians (three prompts, three repetitions). Nsight Compute 2025.3.1 app-range counters [15] use prompt 0, three independent processes and three calls/range, separately from timing. All 112 final counter exports were rechecked.

6 Winner factoring: pretrained two-candidate step

Context	Base ms	Opt. ms	Speedup	Latency lower	Total DRAM change
2048	43.264	31.815	**1.36×**	**26.46%**	−3.91%
3072	63.326	60.386	1.05×	4.64%	+1.24%
4096	82.409	77.858	1.06×	5.52%	−0.27%

This comparison isolates concat winner factoring within the repaired IREE pipeline. All natural and forced-winner cases pass, with exact cache prefixes and unchanged inputs. Worst absolute error: 5.05×10^{-5} (atol= 5×10^{-4}, rtol= 3×10^{-3}). Ordinary decode has no matches. Writes fall but reads rise: full-model DRAM reduction is inconsistent. At context 2048, kernels increase 987→1047/call and logical memcpy remains 188.8 MB/call.

7 Per-transformation operator results

Speedup = baseline time / optimized time; below 1 means slower. CPU rows are separate synthetic studies, not full-model gains. Suite figures are geometric means. Earlier T4 projection timings include output downloads; final model timings leave outputs resident. Combined gains are not additive. Full per-case times, variant names and source paths: `report/study_results/`.

Transformation	Measured effect	Workload / attribution [16]
Original GQA head reuse	1.86× CPU	Default lowering only; no extra gain established over both upstream flags plus fusion.
Conditional read forwarding	2.04× CPU	Cache A1: 187.10→91.74 μs; same best-correct baseline.
Masked gather forwarding	2.52× CPU	Same A1: 187.10→74.36 μs; alternative to conditional reads.
Winner factoring alone	1.38× CPU	2607.34→1885.37 μs; with masked forwarding: 723.42 μs, **3.60× combined**.
Adjacent-update coalescing	0.94–0.96× alone	4–16-token CPU chains. **2.17–2.34× combined** with masked forwarding; merging enables a later match.
Snapshot protection / COW	2.58× combined	109.01→42.27 μs CPU: protection *plus* masked forwarding vs fastest correct baseline; not protection alone.
Key/value projection placement	3.93× CPU; 1.20× T4	16 cases per platform vs strong unabsorbed contractions. One decode ablation: key-only 1.92×, value-only 1.95×.
Partial RoPE + projection	11.40× CPU; 1.50× T4	Four cases per platform; combined pipeline, no isolated RoPE gain. MLA absorption is prior art.
Correct contraction fallback	10.63× CPU	One wide-head case: 131.11→12.33 μs vs correct guarded fused lowering; not a gain from the guard itself.
Gather normalization / native guard	Correctness enablers	No independent valid speedup established. Wrong stock outputs are excluded from performance baselines.
Stable trig reduction	5/9→9/9 cases pass	Earlier matched-valid-case latency overhead: +0.07% (A), +0.25% (B); not a speed optimization.
Extra aggressive fusion flag	0.97× at 2048	Earlier repaired B run: 32.02→33.16 ms; does not consistently help.

8 Forwarding on T4: mechanism and negative control

Track B fixture	Base ms	Opt. ms	Speedup	DRAM MB: base → opt.
Small sparse forks	0.288	0.246	1.17×	5.831→0.128 (97.80% less)
Large sparse forks	1.777	1.230	1.44×	178.897→0.137 (99.92% less)
Dense no-rewrite control	0.545	1.268	**0.43×**	0.285→0.275 (small / variable)

Conditional forwarding, separate from the model factoring result. Track A large fixture: 1.14×, 178.228→0.132 MB in a second T4 session. Synthetic checks use independent FP64 NumPy references (atol= 5×10^{-5}, rtol= 2×10^{-3}). The unexplained no-rewrite regression and uncontrolled clocks limit causal attribution.

Limitations. One FP32 small model and GPU family; custom one-step branching, not complete serving throughput. Logical tensor elimination, physical allocation, DRAM bytes and latency are distinct claims. Incorrect ownership/numerical cases and irregular masked CUDA compile failures remain recorded. No all-GQA, mixed-precision or global-novelty claim is made.

Prior work. GQA, MLA absorption and decoupled RoPE have identifiable originating papers below. Read-over-write has classical foundations. For selection factoring, adjacent-update merging and this exact compiler repair, no definitive first paper is established here. Cache-sharing papers are related systems precedents, not attribution for our precise MLIR patterns.

References and attribution

[1] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón and S. Sanghai. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. EMNLP, 2023, pp. 4895–4901. Originating GQA reference. https://aclanthology.org/2023.emnlp-main.298/

[2] N. Shazeer. *Fast Transformer Decoding: One Write-Head is All You Need*. 2019. Introduces multi-query K/V sharing. https://arxiv.org/abs/1911.02150

[3] J. McCarthy. *Towards a Mathematical Science of Computation*. Information Processing'62, 1962, pp. 21–28. Classical functional-array foundation; not a GQA optimization paper. https://www-formal.stanford.edu/jmc/towards/towards.html

[4] C. Tinelli / SMT-LIB. *ArraysEx: Functional Arrays with Extensionality*. Formal select/store read-over-write axioms. https://smt-lib.org/theories-ArraysEx.shtml

[5] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023. KV sharing and copy-on-write precedent; not our compiler factoring pass. https://arxiv.org/abs/2309.06180

[6] S. Wang, R. Ren and L. Gui. *ForkKV: Scaling Multi-LoRA Agent Serving via Copy-on-Write Disaggregated KV Cache*. 2026. Related branching-cache system, not the first array-law reference. https://arxiv.org/abs/2604.06370

[7] DeepSeek-AI et al. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. 2024. Secs.2.1.2–2.1.3 explicitly describe MLA weight absorption and decoupled RoPE. https://arxiv.org/abs/2405.04434

[8] J. Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. First arXiv version, 2021. Originating RoPE reference. https://arxiv.org/abs/2104.09864

[9] M. H. Payne and R. N. Hanek. *Radian Reduction for Trigonometric Functions*. ACM SIGNUM Newsletter 18(1), 1983, pp. 19–24. Seminal argument-reduction reference; our bounded f64 scheme is not its full algorithm. https://doi.org/10.1145/1057600.1057602

[10] M. Milakov and N. Gimelshein. *Online Normalizer Calculation for Softmax*. 2018. Online normalization background. https://arxiv.org/abs/1805.02867

[11] T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022. Fused IO-aware attention background, not implemented anew in this project. https://arxiv.org/abs/2205.14135

[12] C. Lattner et al. *MLIR: A Compiler Infrastructure for the End of Moore's Law*. 2020. https://arxiv.org/abs/2002.11054

[13] IREE contributors. *GeneralizeLinalgNamedOps; BubbleUpExpandShapes; compilation pipelines*. Source pinned to compiler commit `89ece87b89852100c0a4ac3d515e14842d090967`. Implementation evidence for existing transformations, not a GQA-specific paper. Pinned IREE compiler source; paths `GlobalOptimization/` and `DispatchCreation/`.

[14] Hugging Face. *SmolLM2-135M-Instruct model card and checkpoint*. Pinned revision in the evaluation protocol. https://huggingface.co/HuggingFaceTB/SmolLM2-135M-Instruct

[15] NVIDIA. *Nsight Compute Profiling Guide*. Application-range replay, metric coverage and cache/clock controls. https://docs.nvidia.com/nsight-compute/ProfilingGuide/

[16] CS6886 project. *Attention and KV-cache compiler optimizations: implementation, raw results and reproduction*. Experiment-source snapshot `71cc40ae53c5e60f2a690e67ffbc64e85969e64f`. https://github.com/lazarsalt1441/cs6886_project